Yukti-V: A Pipelined RISC-V Processor with Hardware-Accelerated IEEE-754 Floating-Point Support

Shreevatsa Alawandi
Department of Electronics and Communication Engineering
KLE Technological University
Vidyanagar, Hubballi, Karnataka, India
shreevatsaalawandi@gmail.com
ORCID: 0009-0001-4637-4062

Aishwarya Beedanal
Department of Electronics and Communication Engineering
KLE Technological University
Vidyanagar, Hubballi, Karnataka, India
aishwaryabeedanal2@gmail.com

Chaitra Anantpur
Department of Electronics and Communication Engineering
KLE Technological University
Vidyanagar, Hubballi, Karnataka, India
chaitra.anantpur@kletech.ac.in

***Abstract*—This paper presents the design and FPGA implementation of a five-stage pipelined RISC-V processor integrated with an IEEE-754 single-precision floating-point unit (FPU). The processor follows a classical five-stage pipeline consisting of Fetch, Decode, Execute, Memory Access, and Write Back stages. A hazard management unit is incorporated to mitigate data hazards using forwarding logic, ensuring smooth pipeline operation and reduced stalls. The architecture adopts a Harvard memory organization with separate Instruction Memory (IMEM) and Data Memory (DMEM). The instruction memory supports 128 bytes of instruction storage, while the data memory provides 4 KB of data storage. The floating-point unit supports hardware-based addition, subtraction, multiplication, and division. The addition and subtraction unit performs exponent alignment and priority-based normalization, the multiplication unit incorporates rounding and overflow/underflow handling, and the division unit employs a Newton–Raphson iterative reciprocal approach for efficient computation. The design was developed and synthesized using Vivado 2024.1 and implemented on the Artix-7 Arty A7-100T FPGA board (XC7A100T-FTG256-1). Functional verification was performed using floating-point benchmark programs, including volume and density calculations. Post-synthesis results show a maximum operating frequency of approximately 48.5 MHz with a total on-chip power consumption of 92 mW. The implementation utilizes 1155 LUTs, 447 slice registers, 54 F7 MUXes, and 27 F8 MUXes, demonstrating low hardware overhead. The results confirm that IEEE-754 compliant floating-point integration into a pipelined RISC-V core can be achieved efficiently, making the architecture suitable for embedded and scientific applications requiring floating-point computation.**

***Index Terms*—RISC-V, IEEE 754 Single Precision, Floating Point Unit, Pipelining**

I. Introduction

RISC-V is an open and extensible Instruction Set Architecture (ISA) introduced in 2010 at the University of California, Berkeley, and governed by a global non-profit organization [1]. Unlike proprietary architectures such as x86 and ARM, RISC-V is freely available without licensing restrictions, enabling unrestricted implementation and innovation. It defines a standardized hardware–software interface rather than a specific processor, allowing designers to develop compatible CPUs while retaining architectural flexibility. The ISA is structured around a minimal base instruction set augmented by optional extensions supporting floating-point operations, vector processing, atomic instructions, and advanced memory management.

The significance of RISC-V lies in its modular design and community-driven evolution [1]. Its extensibility enables the development of application-specific and energy-efficient processors across domains ranging from embedded platforms to high-performance computing systems. Furthermore, its open governance model reduces vendor dependency and fosters collaborative advancement of both hardware and software ecosystems. As adoption continues to expand, RISC-V is increasingly regarded as a strategic alternative to established proprietary architectures, offering long-term sustainability and architectural freedom.

A. IEEE 754 Single-Precision Floating-Point Format

The IEEE 754 single-precision (binary32) format represents real numbers using a 32-bit word divided into three fields: sign, exponent, and fraction. The most significant bit denotes the sign (S), where 0 indicates positive and 1 indicates negative. The exponent field consists of 8 bits stored using a bias of 127, such that the actual exponent is obtained as $E - 127$. The remaining 23 bits represent the fraction (mantissa). [2]

For normalized numbers, an implicit leading 1 is assumed before the fractional bits, providing an effective 24-bit significand precision. When the exponent field is zero, the number is classified as subnormal and the implicit leading bit becomes 0, enabling gradual underflow. Special exponent patterns are reserved for infinities and NaN representations. [2]

For normalized values, the represented number is given by:

$$\text{Value} = (-1)^S \times 2^{(E-127)} \times (1 + F) \tag{1}$$

The format provides approximately seven decimal digits of precision and supports a dynamic range of roughly 10^{-38} to

10^{38}, making it suitable for hardware-based floating-point unit implementations. [2] The Bit format is given in the Table I.

TABLE I
IEEE 754 SINGLE-PRECISION (32-BIT) FORMAT

Sign (S)	Exponent (E)	Fraction (F)
1 bit (31)	8 bits (30–23)	23 bits (22–0)

II. LITERATURE SURVEY

Rajyan et al. [3] presented a SystemVerilog-based implementation of an RV32I-compliant RISC-V processor core with a modular structure. The authors designed and integrated essential functional units including the Instruction Fetch unit, Instruction Decode unit, Register File, Arithmetic Logic Unit (ALU), Data Memory, Control Unit, and Branch Control Unit within a unified architecture. The processor followed a single-cycle execution model to simplify instruction flow and control behavior. Extensive simulation and verification were performed to ensure correct instruction execution.

Ahmed et al. [4] developed a five-stage pipelined RV32I RISC-V processor using SystemVerilog and validated the design on the DE2-115 FPGA platform. The architecture adopted a Harvard memory organization and incorporated hazard detection and data forwarding mechanisms to improve instruction throughput. The processor was verified using simulation tools as well as hardware implementation.

Prabhakaran et al. [5] designed a multi-clock domain five-stage pipelined processor based on the RV32I ISA. The study introduced separate clock domains to mitigate timing challenges such as clock skew and metastability across pipeline registers. The processor was implemented in Verilog HDL, simulated using ModelSim, and synthesized on an Altera Cyclone 10 LP FPGA board. Hazard management techniques were incorporated to ensure correct parallel instruction execution, addressing both architectural efficiency and timing stability.

Reddy et al. [6] implemented a five-stage pipelined RISC-V processor using Verilog HDL. The architecture divided execution into Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB) stages, with pipeline registers placed between each stage to maintain controlled data and signal propagation. Forwarding logic and multiplexers were employed to manage data dependencies and reduce unnecessary stalls. Functional verification was performed through simulation to confirm correct execution of arithmetic, logical, and memory operations.

Poli et al. [7] designed a lightweight RISC-V processor intended primarily for educational purposes and FPGA deployment. Unlike pipelined implementations, the processor followed a single-cycle Harvard architecture to reduce structural complexity. The implementation was written in SystemVerilog and deployed on a Basys 3 Artix-7 FPGA board. Performance evaluation using the Dhrystone benchmark demonstrated competitive DMIPS/MHz results while maintaining low hardware resource consumption.

Loh et al. [8] implemented a five-stage pipelined RISC-V processor featuring a complete hazard detection and resolution unit. The processor included mechanisms for data forwarding, pipeline stalling, and flushing to handle both data and control hazards. The design was verified through ModelSim simulation and subsequently synthesized on a Cyclone V DE1-SoC FPGA board.

Tiwari et al. [9] proposed an optimized five-stage pipelined RV32I processor core implemented in Verilog HDL. The design incorporated improvements in the arithmetic logic unit and introduced a pipelined instruction fetch stage combined with dynamic branch prediction. FPGA synthesis and performance analysis demonstrated improved operating frequency compared to baseline implementations.

Lai et al. [10] implemented a 32-bit RV32I-based RISC-V processor using Verilog HDL and verified the design through ModelSim simulation. The architecture followed a structured five-stage pipeline consisting of Instruction Fetch, Instruction Decode, Execute, Memory, and Write-Back stages. The processor was designed to support IoT-oriented embedded applications by maintaining low power consumption and simplified control logic. The study emphasized correct implementation of arithmetic, memory, and branch instructions while validating functional compliance with the RISC-V specification.

Satyanarayana et al. [11] designed a high-performance five-stage pipelined RISC-V processor targeting FPGA implementation. The architecture integrated pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) along with a hazard detection unit to resolve data and control dependencies. The processor also incorporated support for selected RISC-V extensions including floating-point, bit manipulation, and cryptographic operations. Cache memory organization and memory hierarchy considerations were included to improve access efficiency and throughput.

Li et al. [12] developed a five-stage pipelined RISC-V microprocessor with data forwarding and branch prediction mechanisms to manage pipeline hazards. The processor was implemented in Verilog and verified using custom assembly test programs. Performance evaluation included logic synthesis and hardware resource analysis using Vivado tools.

Nguyen-Hoang et al. [13] implemented a 32-bit Linux-capable RISC-V processor integrated with cryptographic accelerators on FPGA and ASIC platforms. The design employed SpinalHDL and LiteX frameworks to configure a VexRiscV core and generate Verilog RTL. Dedicated hardware accelerators for AES-128, SHA-1, and RSA-2048 were incorporated through customized instruction plugins. The processor was deployed on a Nexys4DDR FPGA and later synthesized using a 65nm CMOS process. Functional validation was performed using a modified Linux environment with cryptographic libraries to verify accelerator operation.

Rao et al. [14] designed and implemented a 32-bit single-cycle RISC-V processor using Verilog HDL. The architecture included essential components such as the program counter, register file, arithmetic logic unit, control unit, and memory modules. The implementation supported arithmetic, memory

access, and control-flow instructions aligned with the RV32IM specification.

Karthik et al. [15] integrated a Radix-2 Booth multiplier directly into the execution stage of a pipelined RISC-V processor. Unlike conventional standalone multiplier modules, the Booth multiplier was tightly coupled with the ALU to optimize signed multiplication operations. A control-path optimization strategy was applied to activate the multiplier conditionally, reducing dynamic power consumption. FPGA implementation results demonstrated reduced delay and improved area efficiency. The acceleators and co-processors used in the papers is given in the Figure 1.

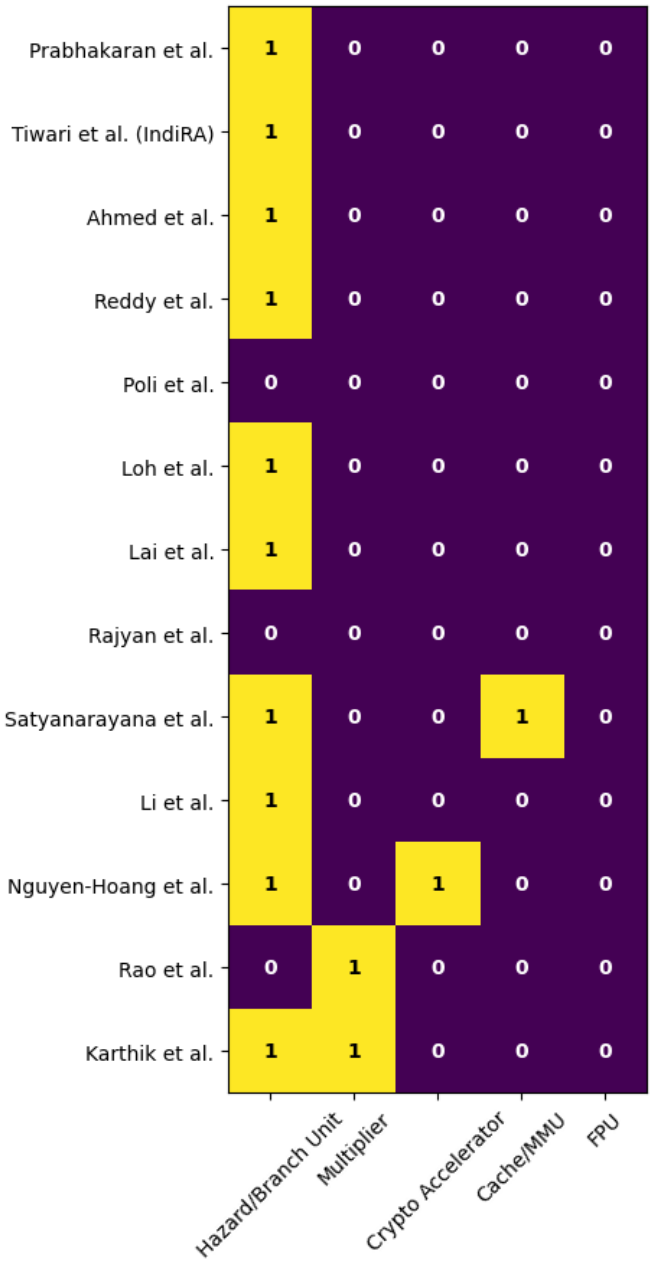

Fig. 1. Accelerators and Co-processors used in the above mentioned works.

A. Research Gap

A comprehensive analysis of the reviewed literature reveals that although several five-stage pipelined RISC-V processors have been implemented and optimized for FPGA deployment, none of the surveyed works incorporate a hardware-based Floating-Point Unit (FPU). Most implementations are restricted to the RV32I base integer instruction set, with some extending support to integer multiplication (RV32IM) or integrating domain-specific accelerators such as cryptographic modules (AES, SHA, RSA) or optimized multipliers (Booth architecture).

While certain studies mention support for floating-point extensions in theory, no implementation provides a dedicated IEEE-754 compliant floating-point datapath, floating-point register file, normalization and rounding logic, or a pipelined floating-point execution unit. As demonstrated in the comparative heatmap analysis (Fig 1), the Floating-Point Unit (FPU) column remains zero across all examined works, confirming the absence of hardware floating-point integration.

This lack of IEEE-754 floating-point support in pipelined RV32I cores represents a clear architectural gap in current FPGA-based academic RISC-V implementations. Modern embedded, signal processing, and scientific workloads increasingly require efficient floating-point computation, making integer-only cores insufficient for many real-world applications.

Motivated by this identified gap, the present work proposes and implements a five-stage pipelined RISC-V processor integrated with a fully functional IEEE-754 single-precision floating-point unit. The design incorporates a dedicated floating-point execution datapath compliant with the IEEE-754 standard, enabling hardware-accelerated floating-point arithmetic while preserving pipeline efficiency and hazard management mechanisms.

III. Processor Architecture

The designed processor uses a five-stage pipelined architecture consisting of Fetch, Decode, Execute, Memory Access, and Writeback stages. Each stage performs a dedicated function and transfers the required information to the next stage through pipeline registers. This structure allows multiple instructions to be processed simultaneously, improving overall throughput and efficiency.

The Fetch stage retrieves instructions from instruction memory. The Program Counter (PC) stores the address of the current instruction and is updated every clock cycle to point to the next sequential instruction. This updated PC value is used to access the instruction memory, and the fetched instruction is stored in the IF/ID pipeline register. The instruction remains in this register until it is forwarded to the Decode stage. Since branch prediction is not implemented in the current floating-point integrated design, the fetch unit operates sequentially without speculative execution logic.

The Decode stage interprets the fetched instruction and generates the necessary control signals for the rest of the pipeline. Immediate values are extended to 32 bits to match the

processor data width. The control logic determines whether the instruction involves arithmetic operations, memory access, or branching. The register file reads the required source operands and prepares them for execution. All relevant data and control signals are stored in the ID/EX pipeline register to maintain synchronization between stages.

The processor additionally includes a dedicated floating-point module compliant with the IEEE-754 single-precision standard. This module supports floating-point addition, subtraction, multiplication, and division. The division operation is implemented using the Newton–Raphson iterative method to improve computational efficiency. The design handles rounding and special cases such as NaN, infinity, and subnormal numbers. The modular structure of the floating-point unit makes it easier to integrate with the main pipeline while maintaining precision and reliability.

In the Memory Access stage, the result from the Execute stage is used either as a memory address (for load and store instructions) or passed forward as a computation result. Load instructions retrieve data from memory, while store instructions write data from the register file into memory. This stage ensures correct data transfer between the processor and memory.

Finally, the Writeback stage updates the register file with the final result of the instruction. A multiplexer selects whether the value to be written comes from memory (for load instructions) or directly from the ALU (for arithmetic and logical instructions). This completes the instruction execution cycle.

To maintain correct operation in a pipelined environment, a hazard management mechanism is included. The hazard detection logic identifies data dependencies between instructions and forwards the most recent results from later pipeline stages to the Execute stage when required. This reduces unnecessary stalls and ensures that dependent instructions receive the correct operands. In cases of severe dependencies, such as when multiple consecutive instructions rely on the same register result, forwarding paths resolve the conflicts efficiently and maintain smooth pipeline operation.

The micro-architecture of the proposed five-stage pipelined RISC-V processor is shown in Fig. 2. The RTL-level structural implementation generated in Vivado 2024.1 is illustrated in Fig. 3.

IV. IEEE-754 Single-Precision Floating-Point Addition and Subtraction Unit

The proposed module implements a 32-bit IEEE-754 single-precision floating-point addition and subtraction unit using a fully combinational architecture. The computation begins by comparing the magnitudes of the two input operands (excluding the sign bit). This ensures that the operand with the larger absolute value is selected as the primary operand, thereby simplifying subtraction and preventing negative intermediate results. The sign, exponent, and mantissa fields are then extracted from both operands. An exception condition is generated if either operand contains an exponent field with all bits equal to one, corresponding to special IEEE-754 values such as infinity or NaN.

The 23-bit mantissa of each operand is extended to a 24-bit significand by restoring the hidden leading bit. For normalized numbers, a leading ‘1‘ is appended, whereas for denormalized numbers, a leading ‘0‘ is used. The exponent difference between the two operands is computed, and the significand of the operand with the smaller exponent is right-shifted accordingly to align both numbers to a common exponent. This alignment is necessary to ensure correct floating-point arithmetic.

Depending on the selected operation and the signs of the operands, the aligned significands are either added or subtracted. In the addition case, the result may generate a carry-out bit. If a carry is produced, the significand is right-shifted by one position and the exponent is incremented to maintain normalized representation. In the subtraction case, two’s complement arithmetic is used to compute the difference between the aligned significands. Since subtraction can introduce leading zeros in the result, normalization is required.

To efficiently normalize the subtraction result, a priority encoder is employed. The priority encoder identifies the position of the most significant ‘1‘ in the subtraction output and determines the required left-shift amount. The significand is then shifted left accordingly, and the exponent is decremented by the same amount to preserve numerical correctness. This approach eliminates iterative shifting and enables fast leading-zero detection in hardware. Finally, the normalized significand, adjusted exponent, and computed sign bit are concatenated to produce the 32-bit IEEE-754 formatted result. If an exception condition is detected, the module asserts the exception flag and forces the output to zero.

The proposed floating-point unit performs IEEE-754 compliant addition and subtraction as detailed in Algorithm 1.

Key Notes:

- S_A, S_B : Sign bits of operands.
- E_A, E_B : 8-bit biased exponents.
- M_A, M_B : 23-bit mantissas.
- $\tilde{M}$: 24-bit significand including hidden bit.
- ΔE : Exponent difference used for alignment.
- Exponent Alignment : Right-shifting the smaller significand to equalize exponents.
- Priority Encoder : Hardware block that detects the leading ‘1’ for normalization.
- Normalization : Left-shifting the result to restore the leading ‘1’ in MSB position.
- Carry Handling : If addition generates overflow, exponent is incremented.
- Exception : Triggered when exponent equals 255 (Infinity/NaN case).

V. IEEE-754 Single-Precision Floating-Point Multiplication Unit

The `Multiplication` module implements a 32-bit single-precision floating-point multiplier compliant with the IEEE-754 standard. The design is entirely combinational and

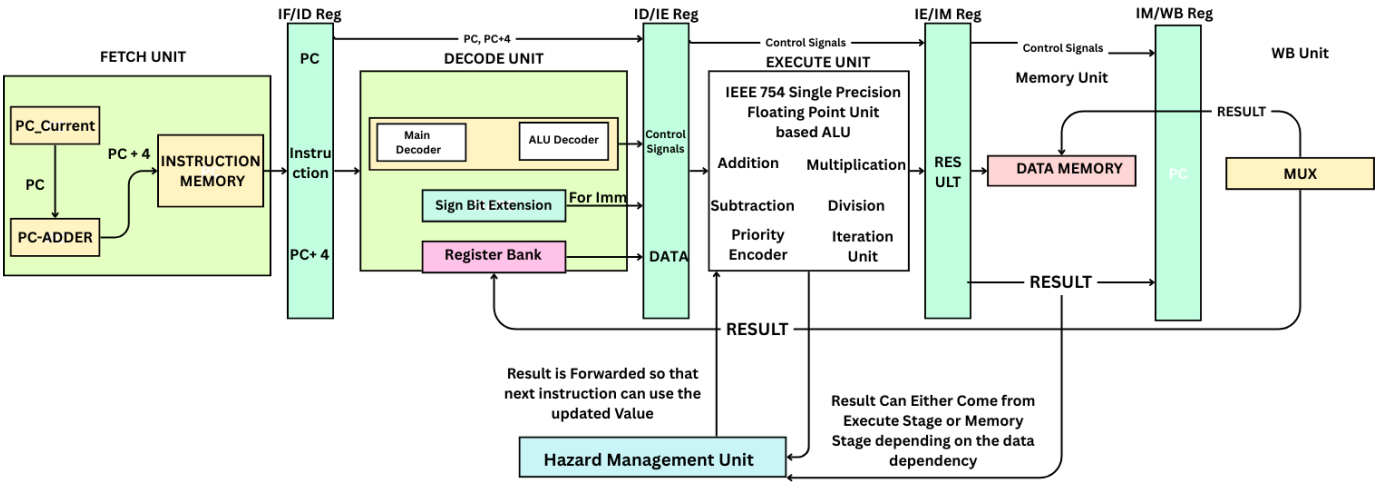

Fig. 2. Micro-architecture of the Processor

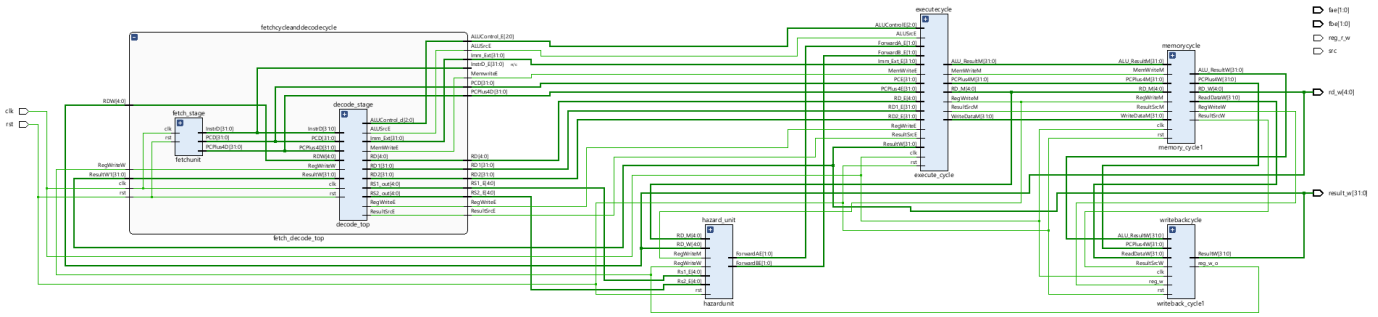

Fig. 3. RTL Design of the processor

decomposes the multiplication process into sign evaluation, exponent arithmetic, significand (mantissa) multiplication, normalization, rounding, and exception handling. Each operand is assumed to follow the IEEE-754 single-precision format, consisting of one sign bit, eight exponent bits with a bias of 127, and twenty-three fraction bits. The module produces the final 32-bit floating-point result along with three status flags: `Exception`, `Overflow`, and `Underflow`.

The multiplication begins with sign computation. In IEEE-754 arithmetic, the sign of a product is determined independently of the magnitude calculation and is simply the exclusive-OR of the input signs. Accordingly, the module computes the result sign by XORing the most significant bits of the two operands. This ensures that the result is positive when both operands share the same sign and negative when their signs differ. Because sign processing is logically independent from exponent and mantissa calculations, it introduces negligible hardware complexity.

The next stage involves detection of exceptional operand conditions. IEEE-754 reserves an exponent value of 255 (all ones) to represent infinities and NaN (Not-a-Number). The module performs a reduction AND operation across the exponent bits of each operand to determine whether either operand has an exponent of 255. If this condition is true for at least one operand, the `Exception` flag is asserted. In this implementation, exceptional inputs cause the output result to be forced to zero. While this simplifies hardware realization, it does not distinguish between infinity and NaN cases, representing a design tradeoff favoring structural simplicity over complete IEEE-754 compliance.

Following exception detection, the significands of the operands are prepared for multiplication. IEEE-754 normalized numbers contain an implicit leading one, commonly referred to as the hidden bit. Therefore, if the exponent field of an operand is nonzero, the module prepends a logical ‘1’ to the 23-bit fraction field, producing a 24-bit significand. Conversely, if the exponent field is zero (indicating a denormalized number), the hidden bit is set to zero. This reconstruction ensures that normalized numbers are treated as 1.Fraction, while denormalized numbers are treated as 0.Fraction. The prepared 24-bit significands are then multiplied, yielding a 48-bit intermediate product.

Exponent processing occurs in parallel with significand multiplication. In floating-point arithmetic, multiplication requires addition of the two biased exponents, followed by subtraction of the bias (127) to restore the correct exponent scaling. The

Algorithm 1 IEEE-754 Single-Precision Floating-Point Addition/Subtraction

Input: 32-bit operands A, B in IEEE-754 format; control $AddBar_Sub$

Output: 32-bit floating-point $Result$, Exception flag

Decompose operands
$S_A \leftarrow A[31]$, $E_A \leftarrow A[30:23]$, $M_A \leftarrow A[22:0]$
$S_B \leftarrow B[31]$, $E_B \leftarrow B[30:23]$, $M_B \leftarrow B[22:0]$
Exception Detection
if $E_A = 255$ or $E_B = 255$ **then**
    $Exception \leftarrow 1$
    return 0
end if
$Exception \leftarrow 0$
Restore Hidden Bit
if $E_A \neq 0$ **then**
    $\tilde{M}_A \leftarrow \{1, M_A\}$
else
    $\tilde{M}_A \leftarrow \{0, M_A\}$
end if
if $E_B \neq 0$ **then**
    $\tilde{M}_B \leftarrow \{1, M_B\}$
else
    $\tilde{M}_B \leftarrow \{0, M_B\}$
end if
Magnitude Comparison
if $|A| < |B|$ **then**
    Swap $(S_A, E_A, \tilde{M}_A)$ with $(S_B, E_B, \tilde{M}_B)$
end if
Exponent Alignment
$\Delta E \leftarrow E_A - E_B$
$\tilde{M}_B \leftarrow \tilde{M}_B >> \Delta E$
Determine Effective Operation
if $AddBar_Sub = 0$ **then**
    $Operation \leftarrow$ Addition
else
    $Operation \leftarrow$ Subtraction
end if
if $Operation =$ Addition **then**
    $M_{sum} \leftarrow \tilde{M}_A + \tilde{M}_B$
    if carry generated **then**
        $M_{sum} \leftarrow M_{sum} >> 1$
        $E_A \leftarrow E_A + 1$
    end if
    $M_{final} \leftarrow M_{sum}[22:0]$
    $S_{final} \leftarrow S_A$
else
    $M_{diff} \leftarrow \tilde{M}_A - \tilde{M}_B$
    Determine leading-one position using priority encoder
    $M_{norm} \leftarrow M_{diff} << shift$
    $E_A \leftarrow E_A - shift$
    $M_{final} \leftarrow M_{norm}[22:0]$
    $S_{final} \leftarrow S_A$
end if
Result Packing
$Result \leftarrow \{S_{final}, E_A, M_{final}\}$

module computes the sum of the two 8-bit exponents using a 9-bit wire to accommodate potential carry-out. The bias correction is then applied, and an additional increment is conditionally added during normalization. The use of a 9-bit exponent register enables detection of overflow and underflow conditions through examination of the most significant bits.

After multiplication of the significands, normalization is required to ensure that the result conforms to IEEE-754 normalized form. Since each operand significand lies within the interval $[1, 2)$, their product lies within $[1, 4)$. Consequently, the 48-bit product may contain a leading '1' in bit position 47 or 46. If the most significant bit (bit 47) is '1', the product is already normalized and no shift is necessary; otherwise, the product is left-shifted by one position to maintain a leading one in the normalized format. A normalization indicator signal is generated and later added to the exponent to reflect any required shift.

Rounding is performed to fit the 48-bit intermediate product back into the 23-bit fraction field. The module extracts bits [46:24] as the candidate mantissa and uses bit 23 as the guard bit. The lower 23 bits are OR-reduced to form a rounding indicator. If the guard bit and any lower discarded bits are high, the mantissa is incremented by one. This mechanism approximates round-to-nearest behavior, although it does not implement full IEEE-754 rounding modes such as ties-to-even. The rounding logic ensures that precision loss due to truncation is minimized.

Zero detection is also incorporated. If the product mantissa becomes zero (after rounding) and no exception is present, the module asserts a `zero` condition. In such cases, the final result is forced to signed zero. This condition prevents propagation of invalid exponent values when the significand collapses to zero.

Overflow and underflow detection are derived from the extended 9-bit exponent result. Overflow occurs when the exponent exceeds the representable 8-bit range after bias adjustment and normalization. The module checks specific high-order bit patterns in the exponent to determine if the value exceeds 255. When overflow is detected and the result is nonzero, the `Overflow` flag is asserted and the output is forced to infinity (represented by an exponent of 255 and a zero mantissa). Underflow occurs when the exponent becomes negative after bias correction, indicating that the result magnitude is too small to be represented as a normalized number. In such cases, the `Underflow` flag is asserted and the result is forced to zero.

Finally, the result is assembled. In normal operation, the output is constructed by concatenating the computed sign bit, the lower eight bits of the adjusted exponent, and the rounded 23-bit mantissa. However, conditional priority logic ensures that exceptional conditions override normal computation. Specifically, exception handling has highest priority, followed by zero detection, overflow, and underflow conditions. The floating point multiplication is given in the algorithm 2.

Algorithm 2 IEEE-754 Single Precision Floating Point Multiplication

```
sign ← A_31 ⊕ B_31
exp_A ← A_30:23, exp_B ← B_30:23
mant_A ← A_22:0, mant_B ← B_22:0
if exp_A = 255 ∨ exp_B = 255 then
    Exception ← 1
else
    Exception ← 0
end if
if exp_A ≠ 0 then
    operand_A ← {1, mant_A}
else
    operand_A ← {0, mant_A}
end if
if exp_B ≠ 0 then
    operand_B ← {1, mant_B}
else
    operand_B ← {0, mant_B}
end if
product ← operand_A × operand_B
if product_47 = 1 then
    normalised ← 1
    product_norm ← product
else
    normalised ← 0
    product_norm ← product ≪ 1
end if
round_bit ← ⋁ product_norm[22 : 0]
product_mantissa ← product_norm[46 : 24] + (product_norm[23] ∧
round_bit)
sum_exp ← exp_A + exp_B
exponent ← sum_exp − 127 + normalised
if product_mantissa = 0 ∧ Exception = 0 then
    zero ← 1
else
    zero ← 0
end if
if exponent > 255 ∧ zero = 0 then
    Overflow ← 1
else
    Overflow ← 0
end if
if exponent < 0 ∧ zero = 0 then
    Underflow ← 1
else
    Underflow ← 0
end if
if Exception = 1 then
    Result ← 0
else if zero = 1 then
    Result ← {sign, 0}
else if Overflow = 1 then
    Result ← {sign, 255, 0}
else if Underflow = 1 then
    Result ← {sign, 0}
else
    Result ← {sign, exponent, product_mantissa}
end if
```

VI. IEEE-754 Single-Precision Floating Point Division Unit

Floating-point division in IEEE-754 single precision format is implemented using a reciprocal-based iterative architecture rather than direct digit-recurrence division. Let the two 32-bit floating-point operands be expressed as:

$$A = (-1)^{S_a} \times 1.M_a \times 2^{E_a - 127} \tag{2}$$

$$B = (-1)^{S_b} \times 1.M_b \times 2^{E_b - 127} \tag{3}$$

The division operation is therefore:

$$\frac{A}{B} = (-1)^{S_a \oplus S_b} \times \frac{1.M_a}{1.M_b} \times 2^{(E_a - E_b)} \tag{4}$$

The proposed architecture decomposes the operation into three stages: sign computation, exponent adjustment, and mantissa division using Newton–Raphson reciprocal approximation.

A. Sign Computation

The sign of the result is determined using a simple XOR operation between the operand sign bits:

$$S_{result} = S_a \oplus S_b \tag{5}$$

This ensures compliance with IEEE-754 sign rules.

B. Exponent Processing

Instead of directly subtracting exponents, the divisor is first normalized to a fixed exponent value of 126 to improve convergence characteristics. The shift value is computed as:

$$Shift = 126 - E_b \tag{6}$$

The adjusted exponent of the dividend becomes:

$$E'_a = E_a + Shift \tag{7}$$

Thus, the effective exponent of the final result is:

$$E_{result} = E'_a - 126 \tag{8}$$

This normalization constrains the divisor mantissa to the range $[0.5, 1)$, which guarantees rapid quadratic convergence of the iterative reciprocal computation.

C. Reciprocal Approximation Using Newton–Raphson Method

The mantissa division is performed by computing the reciprocal of the normalized divisor using the Newton–Raphson iterative method:

$$X_{n+1} = X_n(2 - DX_n) \tag{9}$$

where D represents the normalized divisor mantissa and X_n is the nth approximation of $1/D$.

1) Initial Approximation: A linear polynomial approximation is employed to generate the initial estimate:

$$X_0 = \frac{48}{17} - \frac{32}{17}D \tag{10}$$

This approximation provides sufficient initial accuracy to ensure rapid quadratic convergence.

2) Iterative Refinement: Three Newton–Raphson iterations are performed:

$$X_1 = X_0(2 - DX_0) \tag{11}$$
$$X_2 = X_1(2 - DX_1) \tag{12}$$
$$X_3 = X_2(2 - DX_2) \tag{13}$$

Since Newton–Raphson exhibits quadratic convergence, the number of correct bits approximately doubles after each iteration:

$$|e_{n+1}| \approx C|e_n|^2 \tag{14}$$

After three iterations, full 23-bit mantissa precision required for IEEE-754 single precision is achieved.

D. Final Quotient Computation

Once the reciprocal approximation X_3 is obtained, the final quotient mantissa is computed as:

$$Q = A \times X_3 \tag{15}$$

The final 32-bit IEEE-754 result is constructed as:

$$Result = \{S_{result}, E_{result}, Mantissa\} \tag{16}$$

E. Exception Handling

An exception condition is flagged if either operand has an exponent value of 255:

$$Exception = (E_a = 255) \vee (E_b = 255) \tag{17}$$

Division-by-zero detection can be incorporated by checking:

$$(E_b = 0) \wedge (M_b = 0) \tag{18}$$

F. Hardware Efficiency

The proposed reciprocal-based division unit offers the following architectural advantages:

- Reuse of existing floating-point multiplier and adder/subtractor modules
- Reduced hardware complexity compared to restoring or SRT division
- Quadratic convergence ensuring high precision with limited iterations
- Improved suitability for FPGA implementation

Compared to classical digit-recurrence methods, the Newton–Raphson based architecture provides lower latency and improved pipeline compatibility.

VII. Simulation Results

A. Multiplication Unit

The following RISC-V floating-point assembly program computes the volume of a sphere using the mathematical expression $V = \frac{4}{3}\pi r^3$. Initially, the `flw` instructions load the pre-stored constants $\frac{4}{3} = 1.333$, $\pi = 3.142$, and the radius $r = 5.35$ into floating-point registers `f1`, `f2`, and `f3`, respectively. More Load instructions are inserted as mitigate the hazards caused by the load instruction and are acting as NOP's. The computation phase begins by multiplying `f1` and `f2` using `fmul.s` to obtain $\frac{4}{3}\pi$ in register `f10`. Subsequently, the radius is squared by multiplying `f3` with itself, and then multiplied once more to generate r^3 in `f11`. Finally, the product of $\frac{4}{3}\pi$ and r^3 is computed using another `fmul.s` instruction, and the resulting volume is stored in register `f12`.

Listing 1. RISC-V assembly program for calculating the volume and density of a sphere

```
# Load constants into floating-point registers
flw     f1, 0(x0)       # Load 4/3 = 1.333 into f1
flw     f2, 1(x0)       # Load pi = 3.142 into f2
flw     f3, 2(x0)       # Load r = 5.35 into f3

#Pipeline delay handling(Load instruction as NOP's)
flw    f4, 2(x0)       # NOP
flw    f5, 2(x0)       # NOP
# Compute 4/3 * pi
fmul.s  f10, f1, f2     # f10 = 1.333 * 3.142

# Compute r^2
fmul.s  f11, f3, f3     # f11 = r * r

# Compute r^3
fmul.s  f11, f11, f3    # f11 = r^3

# Final volume calculation
fmul.s  f12, f10, f11   # f12 = (4/3 * pi) * r^3

# Final result stored in f12

# Load mass of the sphere
flw     f13, 3(x0)      # Load mass = 891.234 into
    f13
flw     f14, 3(x0)      # NOP / duplicate load for
    pipeline balancing

# Density computation
fdiv.s  f16, f13, f12   # f16 = mass / volume

# Final density stored in f16
```

The simulation result is illustrated in Figure 4.

B. Division Unit

The volume of the sphere was previously computed using IEEE–754 single-precision floating-point multiplication operations and stored in register `f12`. Following this computation, the program proceeds to evaluate the density of the sphere based on the physical relation $\rho = \frac{m}{V}$. The mass of the sphere, equal to 891.234, is loaded from memory into floating-point register `f13`. To ensure proper pipeline scheduling and to mitigate potential load–use hazards, an additional load instruction is introduced as a buffer. Subsequently, the instruction `fdiv.s f16, f13, f12` performs single-precision floating-point division, where the dividend corresponds to the

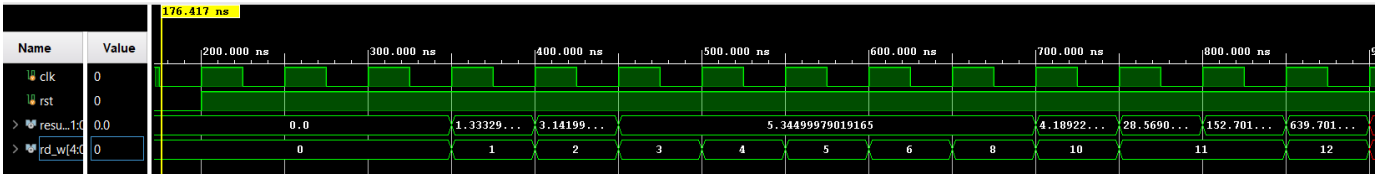

Fig. 4. Simulation Result of the above program. The volume of the sphere is obtained in 12th register. Viewers are adviced to zoom in for clear visibility

mass and the divisor corresponds to the previously computed volume. The resulting density is written into register `f16`.

The simulation result is shown in Figure 5.

VIII. Synthesis Results

A. Maximum Clock Frequency (F_{max})

The design was initially constrained with a clock period of

$$T_i = 200 \text{ ns}. \tag{19}$$

Post-implementation static timing analysis reports a Worst Negative Slack (WNS) of

$$\text{WNS} = +179.371 \text{ ns}, \tag{20}$$

with Total Negative Slack (TNS) equal to 0 ns, indicating that all timing constraints are satisfied.

The effective minimum achievable clock period is computed as

$$T_{\min} = T_i - \text{WNS} \tag{21}$$

$$T_{\min} = 200 - 179.371 \tag{22}$$

$$T_{\min} = 20.629 \text{ ns}. \tag{23}$$

Therefore, the maximum achievable clock frequency is given by

$$F_{\max} = \frac{1}{T_{\min}} \tag{24}$$

$$F_{\max} = \frac{1}{20.629 \times 10^{-9}} \tag{25}$$

Hence, the proposed architecture achieves a maximum operating frequency of approximately

$$\boxed{F_{\max} \approx 48.5 \text{ MHz}}. \tag{26}$$

B. Power Consumption

Power estimation was performed after synthesis using activity derived from constraint files and vectorless analysis. The total estimated on-chip power consumption is summarized in Table II.

The total on-chip power is measured to be

$$P_{\text{total}} = 0.092 \text{ W}. \tag{27}$$

The dynamic power consumption is

$$P_{\text{dynamic}} = 0.001 \text{ W}, \tag{28}$$

which accounts for approximately 1% of the total power.

The dominant contribution is from device static power,

$$P_{\text{static}} = 0.091 \text{ W}, \tag{29}$$

which represents nearly 99% of the total on-chip power. This indicates that leakage power dominates over switching activity at the current operating frequency.

The junction temperature is reported as 25.4°C, with an ambient temperature of 25°C and an effective thermal resistance of 4.6°C/W. The available thermal margin is 59.6°C, indicating safe thermal operation.

TABLE II
On-Chip Power Summary

Component	Power (W)	Percentage
Clocks	<0.001	24%
Signals	<0.001	17%
Logic	<0.001	20%
DSP	<0.001	<1%
I/O	<0.001	38%
Total Dynamic Power	0.001	1%
Device Static Power	0.091	99%
Total On-Chip Power	0.092	100%

C. FPGA Resource Utilization

The post-synthesis resource utilization of the proposed `TOP_RISC_V` architecture is summarized in Tables III and IV. The target FPGA device provides 63,400 Slice LUTs, 12,680 Slice Registers, 31,700 F7 MUXes, 15,850 F8 MUXes, 240 DSP blocks, 170 Bonded IOBs, and 32 BUFGCTRL resources.

TABLE III
Overall FPGA Resource Utilization

Resource	Used	Available	Utilization
Slice LUTs	1155	63400	2%
Slice Registers	447	12680	1%
F7 MUXes	54	31700	<1%
F8 MUXes	27	15850	<1%
DSPs	2	240	1%
Bonded IOB	45	170	26%
BUFGCTRL	1	32	3%

IX. Conclusion and Future Research

This work presented the design and FPGA implementation of a five-stage pipelined RISC-V processor integrated with an IEEE-754 single-precision floating-point unit. Unlike many academic RV32I implementations that support only integer operations, the proposed architecture enables hardware-based

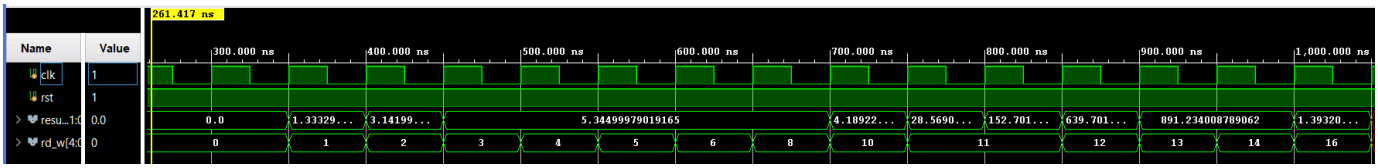

Fig. 5. Simulation Result of the program for calculating the density of the sphere.

TABLE IV
HIERARCHICAL MODULE-WISE RESOURCE UTILIZATION

Module	LUTs	Registers	F7	F8	DSPs
TOP_RISC_V	1155	447	54	27	2
execute_cycle	163	65	0	0	2
fetch_decode	770	317	0	0	0
memory_cycle	211	65	54	27	0
writeback_cycle	19	0	0	0	0

floating-point addition, subtraction, multiplication, and division. Simulation results validated correct execution of floating-point programs, including volume and density calculations, while synthesis results demonstrated low resource utilization, modest power consumption, and stable timing performance with a maximum operating frequency of approximately 48.5 MHz. These results confirm that integrating floating-point capability into a pipelined RISC-V core can be achieved efficiently without significant hardware overhead.

Future work may focus on improving IEEE-754 compliance by incorporating additional rounding modes and comprehensive exception handling. Architectural enhancements such as pipelined floating-point execution, fused multiply–add (FMA) support, branch prediction, and cache integration could further improve performance. Extending the design to support double-precision arithmetic and evaluating the architecture on ASIC platforms would provide deeper insight into scalability and real-world applicability.

REFERENCES

[1] N. Brown, “What is RISC-V and why should we care?,” in *Proceedings of the Cray User Group (CUG ’25)*, New York, NY, USA: Association for Computing Machinery, 2025, pp. 178–189. Available: https://doi.org/10.1145/3757348.3757367

[2] “Single-precision floating-point format,” Wikipedia, The Free Encyclopedia, Feb. 2026. [Online]. Available: https://en.wikipedia.org/wiki/Single-precision_floating-point_format. [Accessed: Feb. 17, 2026].

[3] A. Rajyan and G. Saini, ”SystemVerilog Based Design of an RV32I Compliant RISC-V Processor Core,” in *2024 5th IEEE Global Conference for Advancement in Technology (GCAT)*, Bangalore, India, 2024, pp. 1–5. doi: 10.1109/GCAT62922.2024.10923874.

[4] S. Ahmed and A. B. M. Harun-Ur-Rashid, ”Design, Implementation and Verification of Five Stage Pipeline RISC-V Core (RV32I ISA),” in *2025 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, Chittagong, Bangladesh, 2025, pp. 1–6. doi: 10.1109/ECCE64574.2025.11013913.

[5] S. Prabhakaran, M. N and V. Vedanarayanan, ”Design and Analysis of a Multi Clocked Pipelined Processor Based on RISC-V,” in *2022 International Conference on Communication, Computing and Internet of Things (IC3IoT)*, Chennai, India, 2022, pp. 1–5. doi: 10.1109/IC3IOT53935.2022.9767960.

[6] L. R. Reddy, B. H. Reddy, B. K. Reddy, B. Deekshith Goud and U. P. Lakshmi, ”Five Stage Pipelined RISC V Processor using Verilog,” in *2025 9th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, Coimbatore, India, 2025, pp. 1498–1501. doi: 10.1109/ICECA66444.2025.11383538.

[7] L. Poli, S. Saha, X. Zhai and K. D. Mcdonald-Maier, ”Design and Implementation of a RISC V Processor on FPGA,” in *2021 17th International Conference on Mobility, Sensing and Networking (MSN)*, Exeter, United Kingdom, 2021, pp. 161–166. doi: 10.1109/MSN53354.2021.00037.

[8] S. H. Loh, G. Y. Tan and J. J. Sim, ”VLSI Design Course with FPGA Implementation of Pipelined RISC-V Processor,” in *2025 IEEE 15th International Conference on Control System, Computing and Engineering (ICCSCE)*, Batu Ferringhi, Penang, Malaysia, 2025, pp. 36–41. doi: 10.1109/ICCSCE65566.2025.11182670.

[9] A. Tiwari, P. Guha, G. Trivedi, N. Gupta, N. Jayaraj and J. Pidanic, ”IndiRA: Design and Implementation of a Pipelined RISC-V Processor,” in *2023 33rd International Conference Radioelektronika (RADIOELEKTRONIKA)*, Pardubice, Czech Republic, 2023, pp. 1–6. doi: 10.1109/RADIOELEKTRONIKA57919.2023.10109058.

[10] J.-Y. Lai, C.-A. Chen, S.-L. Chen and C.-Y. Su, ”Implement 32-bit RISC-V Architecture Processor using Verilog HDL,” in *2021 International Symposium on Intelligent Signal Processing and Communication Systems (ISPACS)*, Hualien City, Taiwan, 2021, pp. 1–2. doi: 10.1109/ISPACS51563.2021.9651130.

[11] B. V. V. Satyanarayana, N. Madhu and K. Srinivas, ”Design and Implementation of High-Performance RISC-V Processor with Five Stage Pipeline on FPGA,” in *2025 3rd International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*, Erode, India, 2025, pp. 68–75. doi: 10.1109/ICSSAS66150.2025.11081376.

[12] Y. Li, Z. Yu and W. Ji, ”Design and Performance Optimization of a Risc-V Processor with Five-Stage Pipeline,” in *2025 IEEE 6th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT)*, Shenzhen, China, 2025, pp. 1–5. doi: 10.1109/AINIT65432.2025.11035005.

[13] D.-T. Nguyen-Hoang, K.-M. Ma, D.-L. Le, H.-H. Thai, T.-B.-T. Cao and D.-H. Le, ”Implementation of a 32-Bit RISC-V Processor with Cryptography Accelerators on FPGA and ASIC,” in *2022 IEEE Ninth International Conference on Communications and Electronics (ICCE)*, Nha Trang, Vietnam, 2022, pp. 219–224. doi: 10.1109/ICCE55644.2022.9852060.

[14] M. R. P, P. Niranjan and D. K. M. J, ”Design and Implementation of 32-bit RISC-V Processor Using Verilog,” in *2024 IEEE International Conference on Distributed Computing, VLSI, Electrical Circuits and Robotics (DISCOVER)*, Mangalore, India, 2024, pp. 181–186. doi: 10.1109/DISCOVER62353.2024.10750638.

[15] K. E and K. Singh, ”Implementation of RISC-V Processor along with Radix-2 Booth Multiplier Using Verilog,” in *2025 First International Conference on Intelligent Computing and Communication Systems (CICCS)*, Bengaluru, India, 2025, pp. 1–5. doi: 10.1109/CICCS66437.2025.11280057.